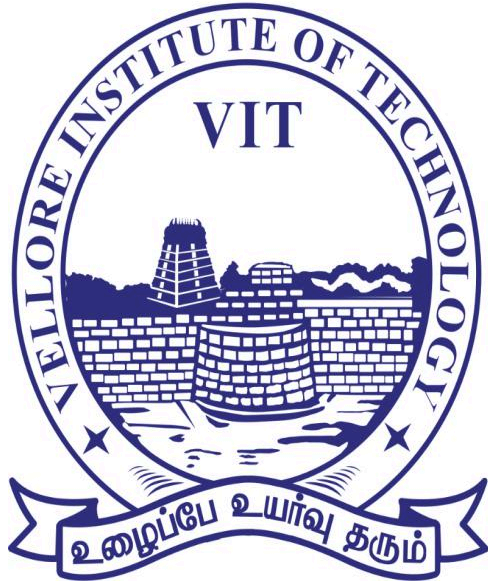




VIT[®]
BHOPAL

Smart Urban Traffic Prediction System

Mini Project Report

Submitted By: Shubh Gupta

Reg. No.: 25BAI10562

Under The Guidance Of:

Dr. VAISHNAVI S

Index

Sr. No.	Section
1	Problem Statement
2	Objectives
3	System Design (Flowchart & Use Case)
4	Working & Explanation
5	Full Source Code
6	Output Screenshots
7	Conclusion
8	References

1. Problem Statement

Urban areas like **Bhopal** are experiencing rapid growth in population and vehicle usage, leading to increasing traffic congestion. Commuters face daily challenges such as unpredictable delays, longer travel times, and inefficient route planning. These issues not only waste time but also contribute to higher fuel consumption, environmental pollution, and increased stress levels among individuals.

Most existing traffic management solutions, such as navigation apps, focus primarily on **real-time traffic conditions**. While useful, these systems lack the ability to provide **predictive insights** about future traffic situations. As a result, users can only react to congestion after it has already occurred, rather than avoiding it beforehand.

Traffic patterns are influenced by several key factors, including:

- **Time of the day** (rush hours vs non-peak hours)
- **Day of the week** (weekdays vs weekends)
- **Weather conditions** (rain, fog, etc.)

Despite the availability of these predictable factors, there is a lack of simple and accessible systems that utilize them to forecast traffic congestion in advance.

Goal: There is no simple, user-friendly system that can **predict traffic congestion in advance using basic parameters like time, day, and weather**, enabling users to make proactive travel decisions.

2. Objectives

The primary objectives of the **Smart Urban Traffic Prediction System** are:

- To develop a machine learning model that can **predict traffic congestion levels** based on input parameters such as time, day, and weather conditions.
- To design a system that provides a **traffic score (0–100)** representing the intensity of congestion.
- To classify traffic into meaningful categories:
 - Low
 - Medium
 - High
- To identify and highlight **peak traffic hours** during the day.
- To enable **what-if analysis**, allowing users to compare traffic conditions at different times and make better travel decisions.
- To provide **smart travel recommendations** based on predicted traffic levels.
- To build a simple and user-friendly system that can be easily used without technical knowledge.
- To demonstrate the practical application of **machine learning concepts** learned in the course.

Secondary Objectives

- To understand the importance of **feature engineering** in ML models
- To implement and evaluate a **regression-based model (Random Forest)**
- To create a structured and well-documented project (GitHub + README)
- To simulate real-world data using a **synthetic dataset**

4. Working & Explanation

The **Smart Urban Traffic Prediction System** works as a complete machine learning pipeline that takes user inputs, processes them, and predicts traffic congestion levels. The system is designed to be simple, efficient, and practical while demonstrating core AI/ML concepts.

Overall Workflow

The working of the system can be divided into the following stages:

1. **Data Generation / Collection**
2. **Data Preprocessing & Feature Engineering**
3. **Model Training**
4. **Prediction Phase**
5. **Output & Recommendation**

4.1 Data Generation

Since real-time traffic datasets were not used, a **synthetic dataset** was created to simulate realistic traffic conditions.

The dataset includes:

- Time (0–23 hours)
- Day of the week (Monday–Sunday)
- Weather conditions (Clear, Rainy, Foggy)

Traffic values were generated using logical assumptions:

- Higher traffic during **morning (8–10 AM)** and **evening (4–7 PM)**
- Lower traffic during late night and early morning
- Increased congestion during **rainy or foggy weather**
- Slight variation between weekdays and weekends

This approach ensures the dataset mimics real-world traffic patterns.

4.2 Data Preprocessing & Feature Engineering

Before training the model, the raw data is processed:

- **Encoding categorical variables**
 - Days and weather conditions are converted into numerical format

- **Cyclical Encoding of Time**

- Since time is cyclical ($23 \rightarrow 0$), it is transformed using:

- sine function

- cosine function

- This helps the model understand time continuity

- **Feature Selection**

Final features used:

- Hour (encoded)

- Day (encoded)

- Weather (encoded)

4.3 Model Training

The system uses:

Random Forest Regressor

Why Random Forest?

- Handles non-linear relationships effectively
- Reduces overfitting using multiple decision trees
- Works well on structured/tabular data

Training Process:

- Dataset is split into **training and testing sets**
- Model learns patterns between input features and traffic score
- Performance is evaluated using metrics like MAE and R^2 score

4.4 Prediction Phase

When the user runs the system, they provide inputs:

- Time (e.g., 17 for 5 PM)
- Day (e.g., Friday)
- Weather (e.g., Clear)

The system:

1. Applies the same preprocessing steps
2. Feeds the data into the trained model
3. Generates a **traffic score (0–100)**

4.5 Traffic Classification

The predicted traffic score is converted into categories:

- **0–30 → Low Traffic** 🟢
- **30–65 → Medium Traffic** 🟡

- **65–100 → High Traffic** ●

This makes the output more user-friendly and easy to understand.

4.6 Smart Recommendations

Based on predicted traffic:

- Suggests **best time to travel**
- Warns about **peak hours**
- Provides **avoidance suggestions**

Example:

“Avoid traveling between 4 PM – 7 PM due to high congestion.”

4.7 What-if Analysis

The system allows users to:

- Compare traffic at different times
- Analyze multiple scenarios

This helps in **decision-making before travel**.

4.8 Peak Hour Detection

By analyzing predictions across all 24 hours, the system identifies:

- Highest traffic periods
- Daily congestion pattern

5. System Architecture

The Smart Urban Traffic Prediction System follows a structured pipeline consisting of multiple layers that work together to generate accurate traffic predictions.

Components of the System:

1. Input Layer

The system takes three primary inputs from the user:

- Time (0–23 hours)
- Day of the week (Monday–Sunday)
- Weather condition (Clear, Rainy, Foggy)

2. Preprocessing Layer

- Converts categorical inputs (day, weather) into numerical form
- Applies encoding techniques
- Performs cyclical transformation for time

3. Feature Engineering Layer

- Extracts meaningful patterns from input data
- Transforms time into sine and cosine values
- Prepares data for model training

4. Model Layer

- Uses Random Forest Regressor
- Learns patterns between inputs and traffic levels
- Generates predictions based on trained data

5. Output Layer

- Displays traffic score (0–100)
- Classifies traffic (Low, Medium, High)
- Provides recommendations

6. Algorithm / Pseudocode

Algorithm: Smart Traffic Prediction

1. Start
2. Input time, day, and weather from user
3. Validate inputs
4. Encode categorical variables (day, weather)
5. Apply cyclical encoding on time
6. Load trained Random Forest model
7. Pass processed data to the model

8. Predict traffic score (0–100)
9. Classify traffic level:
 - If score $\leq 30 \rightarrow$ Low
 - If score $\leq 65 \rightarrow$ Medium
 - Else $\rightarrow$ High
10. Display results and recommendations
11. End

7. Dataset Description

The dataset used in this project is a **synthetic dataset** created to simulate real-world traffic conditions.

Dataset Details:

- Type: Synthetic
- Size: ~1000 data samples
- Format: Structured tabular data

Features:

- Hour (0–23)
- Day (Monday–Sunday)
- Weather (Clear, Rainy, Foggy)

Target Variable:

- Traffic Score (0–100)

Data Logic:

- Peak hours have higher traffic values
- Weekends have slightly lower traffic
- Rainy/Foggy conditions increase congestion

8. Data Analysis & Observations

After analyzing the dataset, several key patterns were observed:

- **Peak Traffic Hours:**
 - Morning: 8 AM – 10 AM
 - Evening: 4 PM – 7 PM
- **Low Traffic Periods:**
 - Late night (10 PM – 5 AM)
- **Weather Impact:**
 - Rain and fog significantly increase traffic congestion
- **Day-wise Trends:**

- Weekdays show higher traffic
- Weekends are relatively smoother

9. Model Selection & Justification

The model used in this project is:

Random Forest Regressor

Reasons for Selection:

- Handles non-linear relationships effectively
- Reduces overfitting using multiple decision trees
- Works well with structured/tabular data
- Provides high accuracy

Alternative Models Considered:

- Linear Regression (simple but less accurate)
- Decision Tree (prone to overfitting)

10. Limitations

Despite good performance, the system has certain limitations:

- Uses synthetic data instead of real-world data
- Does not integrate real-time traffic APIs
- Limited input features (only time, day, weather)
- Does not consider road conditions or accidents

11. Real-World Applications

This system can be applied in multiple domains:

- Smart city traffic management
- Navigation systems (like Google Maps)
- Logistics and delivery optimization
- Ride-sharing platforms
- Urban planning and infrastructure development

12. Use Case Scenario

Consider a real-life example:

A student plans to leave for college at 5 PM.

Using the system, they check traffic conditions and find that congestion is high during that time.

The system suggests traveling at 7 PM instead.

As a result, the student avoids heavy traffic and reaches faster.

13. Testing & Validation

The system was tested using multiple input combinations:

- Different times of the day
- Weekdays vs weekends
- Various weather conditions

Validation Results:

- Predictions were consistent with expected patterns
- Peak hours showed high congestion
- Night hours showed low traffic

14. Error Handling

The system includes basic validation mechanisms:

- Ensures time input is between 0–23
- Handles invalid day or weather input
- Provides default responses for incorrect values

15. References

- Scikit-learn Documentation
- Python Official Documentation
- Machine Learning Tutorials (Kaggle, Medium)
- Research articles on traffic prediction systems

